

TRAVEL BUSINESS SaaS

Complete Backend Master Plan & Product Specification

Product Type: Multi-Tenant Travel Business Operating System

Target Market: Bangladesh-first, globally scalable

Primary Users: Tour Operators, Travel Agencies, DMCs, Corporate Tour Organizers, Group Tour Businesses

Backend: Nodejs + TypeScript

Database: PostgreSQL + Prisma

Cache / Queue: Redis + BullMQ

Storage: Supabase Storage / S3-compatible storage

API: REST API, versioned

Architecture: Multi-tenant SaaS

1. PRODUCT VISION

Build a complete cloud-based Travel Business Operating System that allows travel companies to manage their entire business from one platform.

The system must combine:

- Tour Management
- Online Booking
- Customer CRM
- Traveler Management
- Tour Operations
- Transportation
- Hotels
- Guides
- Corporate Tours
- Custom Tours
- Sales & Leads
- Quotations
- Payments
- Invoices
- Expenses
- Accounting
- Cash Management
- Bank Management
- Vendor Management

- Commission
- Profit & Loss
- Reports
- Website CMS
- Online Booking Engine
- Notifications
- Staff Management
- Role & Permission
- Multi-tenant SaaS
- Subscription Billing
- API & Integrations

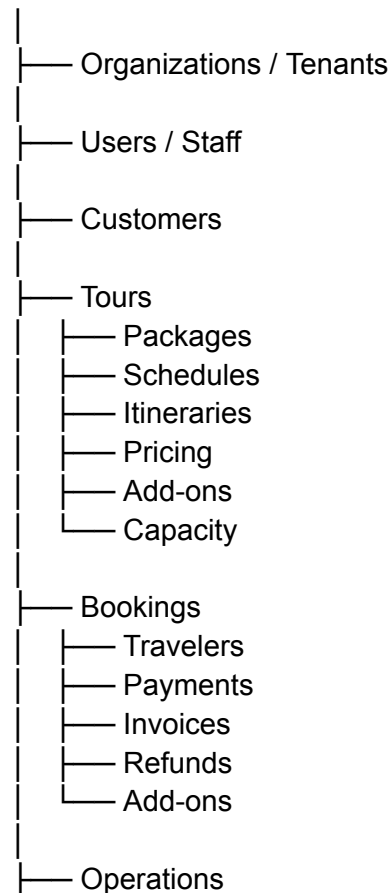
The goal is not to build a simple admin panel.

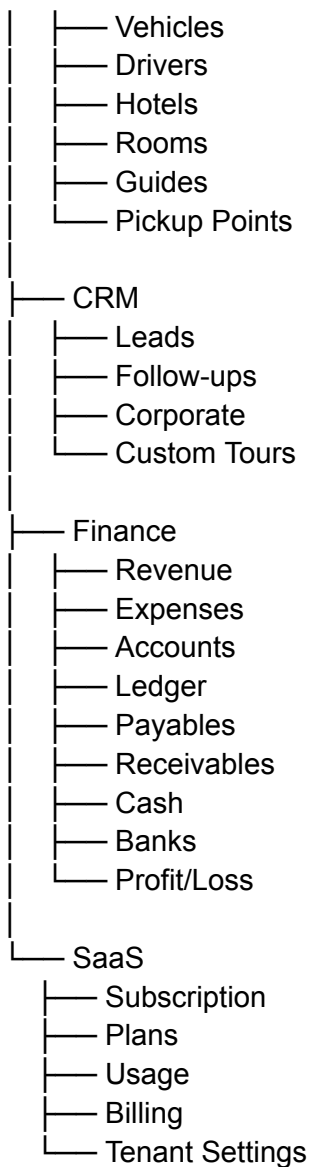
The goal is to build a reusable SaaS platform where hundreds or thousands of independent travel companies can operate their businesses.

2. CORE PRODUCT MODEL

The system hierarchy:

SaaS Platform





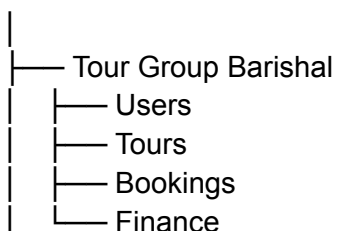
3. MULTI-TENANT ARCHITECTURE

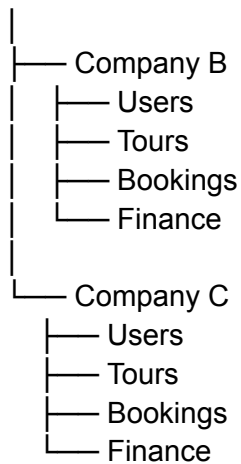
This is one of the most important architectural decisions.

Every business must be an independent tenant.

Example:

Platform





No tenant may access another tenant's data.

Every tenant-owned table must have:

organizationId

Use PostgreSQL Row Level Security where appropriate.

Backend authorization must also validate tenant ownership.

Never rely only on frontend filtering.

4. SAAS PLATFORM ADMIN

Separate platform-level administration from tenant administration.

Platform Admin

Can manage:

- Organizations
- Subscription plans
- Subscriptions
- Billing
- Platform users
- Feature flags
- System settings
- Platform analytics
- Tenant status
- Support tickets
- System announcements
- Audit logs

- API usage
 - Storage usage
 - Notification usage
-

5. ORGANIZATION / TENANT MANAGEMENT

Each business has:

- Organization ID
- Name
- Legal name
- Slug
- Logo
- Favicon
- Email
- Phone
- WhatsApp
- Address
- Country
- Currency
- Timezone
- Website
- Domain
- Business type
- Status
- Subscription
- Created date

Business types:

- Tour Operator
 - Travel Agency
 - DMC
 - Corporate Travel
 - Group Tour Organizer
 - Hotel + Tour
 - Other
-

6. TENANT ONBOARDING

Create onboarding flow:

Create Account



Create Organization



Select Business Type



Company Information



Branding



Currency / Timezone



First Tour



Payment Settings



Invite Staff



Complete

Track onboarding progress.

7. SAAS SUBSCRIPTION MANAGEMENT

Plans:

Starter

- Limited staff
- Tour management
- Booking
- Customer CRM
- Basic finance

Business

- More users
- Advanced CRM
- Full finance
- Reports
- Website booking
- Notifications
- Operations

Enterprise

- Unlimited users
- Multiple branches
- White-label
- Custom domain
- Advanced accounting
- API
- Custom integrations
- Priority support

Plan features must be database-driven.

Do not hardcode plan limitations.

8. SUBSCRIPTION BILLING

Support:

- Monthly
- Yearly
- Trial
- Upgrade
- Downgrade
- Cancel
- Renew
- Grace period
- Failed payment
- Suspended subscription

Track:

- Subscription ID
 - Organization
 - Plan
 - Start date
 - Renewal date
 - Status
 - Amount
 - Payment provider
 - Invoice
-

9. FEATURE ENTITLEMENT SYSTEM

Each plan can have features:

tour_management
booking
crm
finance
corporate
custom_tour
advanced_reports
white_label
custom_domain
api_access
multiple_branches

Backend middleware checks entitlement.

Example:

```
organization.hasFeature("advanced_reports")
```

10. USER MANAGEMENT

Users belong to an organization.

Fields:

- Name
- Email
- Phone
- Avatar
- Status
- Password
- Last login
- Role

Support:

- Invite user
- Accept invitation
- Remove user
- Suspend user
- Restore user
- Reset password
- Change password

11. ROLE MANAGEMENT

Default roles:

- Owner
- Super Admin
- Admin
- Booking Manager
- Tour Manager
- Finance Manager
- Accountant
- Sales
- Support
- Content Manager
- Operations Staff
- Guide

Custom roles must be supported.

12. PERMISSION SYSTEM

Granular permission model.

Examples:

tour.create
tour.read
tour.update
tour.delete

booking.create
booking.read
booking.update
booking.confirm
booking.cancel

payment.read
payment.create
payment.verify
payment.refund

expense.create
expense.approve

report.read

Support:

- Role permissions
 - User-specific permissions if necessary
 - Module permissions
 - Action permissions
-

13. AUTHENTICATION

Implement:

- Login
- Logout
- Access token
- Refresh token
- Password reset
- Email verification
- OTP
- Phone verification
- Session management
- Device tracking
- Optional 2FA

Use secure password hashing.

Never store plaintext passwords.

14. SECURITY

Implement:

- Authentication
- Authorization
- RBAC
- Tenant isolation
- RLS
- Input validation

- Rate limiting
- CORS
- Helmet/security headers
- Secure cookies where applicable
- CSRF protection where applicable
- Request size limits
- File upload validation
- SQL injection protection through ORM
- Audit logging
- Secret management
- Environment variables

Never expose:

- Payment secrets
- API keys
- Database credentials
- Internal errors
- Private documents

15. DESTINATION MANAGEMENT

Destination entity:

- Name
- Slug
- Country
- Division
- District
- Upazila
- Coordinates
- Description
- Images
- Status
- Featured

Support destination hierarchy.

16. TOUR CATEGORY

Categories:

- Day Tour
- Family Tour
- Adventure
- Corporate
- Educational
- Promotional
- Relaxation
- Group Tour
- Custom
- International
- Other

Tenant-specific categories allowed.

17. TOUR PACKAGE MANAGEMENT

Fields:

- Tour ID
- Tour code
- Name
- Slug
- Description
- Short description
- Destination
- Category
- Duration
- Cover image
- Gallery
- Status
- Featured
- Booking deadline
- Minimum travelers
- Maximum travelers

Status:

DRAFT
PUBLISHED
ARCHIVED

18. TOUR SCHEDULE MANAGEMENT

One package can have multiple schedules.

Example:

Sundarbans Tour

03 Sep

10 Sep

17 Sep

24 Sep

Each schedule independently controls:

- Date
- Time
- Capacity
- Price
- Booking deadline
- Departure
- Availability
- Status

19. CAPACITY / INVENTORY ENGINE

Track:

- Total capacity
- Available seats
- Pending seats
- Confirmed seats
- Reserved seats
- Blocked seats

Never calculate availability only on frontend.

Backend must be authoritative.

20. CONCURRENCY / OVERBOOKING PROTECTION

Critical requirement.

When multiple users book simultaneously:

BEGIN TRANSACTION

Lock schedule capacity

Check available seats

Reserve seats

Create booking

Update inventory

COMMIT

If insufficient capacity:

Rollback.

Never allow negative capacity.

21. SEAT HOLD SYSTEM

Support temporary reservations.

Fields:

- Schedule
- Number of seats
- Customer
- Expiry
- Reason
- Created by

Expired holds automatically release seats.

Use Redis/BullMQ or scheduled worker.

22. TOUR PRICING ENGINE

Pricing must be server-side.

Support:

- Adult price
- Child price
- Infant price
- Group pricing
- Seasonal pricing
- Schedule-specific pricing
- Corporate pricing
- Custom pricing

Price calculation must be reproducible.

Store price snapshot inside booking.

Future tour price changes must not alter historical booking totals.

23. ITINERARY MANAGEMENT

Support:

- Day
- Time
- Location
- Activity
- Description
- Meal
- Transport
- Hotel
- Ordering

Allow versioning if itinerary changes after bookings.

24. INCLUSION / EXCLUSION

Manage:

Included

- Transport
- Food
- Hotel

- Guide
- Entry fee

Excluded

- Personal expenses
- Shopping
- Optional activities

All configurable.

25. TOUR ADD-ONS

Examples:

- Extra meal
- VIP seat
- Hotel upgrade
- Private pickup
- Extra luggage
- Optional activity

Fields:

- Name
 - Description
 - Price
 - Quantity
 - Availability
 - Schedule applicability
-

26. BOOKING ENGINE

Booking sources:

- Website
- Admin
- Phone
- Office
- Facebook
- WhatsApp
- Referral

- Agent
- Corporate

Booking must create:

- Booking record
 - Customer
 - Travelers
 - Price snapshot
 - Seat allocation
 - Payment state
 - Booking history
-

27. BOOKING REFERENCE

Format:

TBS-2026-000001

Must be unique per platform or tenant according to configuration.

Invoice:

INV-2026-000001

Payment:

PAY-2026-000001

Refund:

REF-2026-000001

Quotation:

QUO-2026-000001

28. BOOKING STATUS

PENDING

AWAITING_PAYMENT

PAYMENT_SUBMITTED

CONFIRMED

PARTIALLY_PAID

PAID
CANCELLED
REFUND_REQUESTED
REFUNDED
COMPLETED
NO_SHOW

Track every transition.

29. BOOKING STATUS HISTORY

Every status change:

- Old status
- New status
- User
- Timestamp
- Reason
- Note

Never overwrite history.

30. CUSTOMER CRM

Customer:

- Name
- Phone
- WhatsApp
- Email
- Address
- Emergency contact
- Source
- Tags
- Notes

Analytics:

- Total bookings
- Completed trips
- Cancelled trips
- Total spent

- Outstanding
 - Lifetime value
 - Last booking
-

31. TRAVELER MANAGEMENT

One booking can contain multiple travelers.

Traveler:

- Name
- Age
- Gender
- Phone
- Emergency contact
- Special requirements
- Food preference
- Optional identification information

Sensitive identification information must be optional and securely protected.

32. PICKUP POINT

Manage:

- Name
- Address
- Contact
- Time
- Capacity
- Status

Assign pickup per booking/traveler.

33. TOUR MANIFEST

Generate:

- Tour
- Schedule
- Traveler list
- Booking ID
- Phone
- Pickup
- Payment status
- Notes

Support:

- CSV
 - Excel
 - PDF
 - Print
-

34. TRANSPORT MANAGEMENT

Vehicles:

- Bus
- Microbus
- Car
- Launch
- Boat

Vehicle:

- Name
 - Registration
 - Type
 - Capacity
 - Owner
 - Status
 - Maintenance information
-

35. DRIVER MANAGEMENT

Driver:

- Name
- Phone

- License
- Status
- Emergency contact
- Notes

Assign driver to tour schedule.

36. TRANSPORT ASSIGNMENT

Tour Schedule

↓

Vehicle

↓

Driver

↓

Helper

Prevent assigning unavailable vehicles.

37. TRANSPORT COST

Track:

- Rent
- Fuel
- Toll
- Parking
- Driver payment
- Helper payment
- Boat cost
- Other

Link expenses to tour/schedule.

38. HOTEL MANAGEMENT

Hotel:

- Name

- Location
 - Contact
 - Address
 - Rating
 - Amenities
 - Status
-

39. ROOM MANAGEMENT

Room:

- Room number
- Room type
- Capacity
- Price
- Status

Allocation:

Tour

→ Hotel

→ Room

→ Travelers

40. GUIDE MANAGEMENT

Guide:

- Name
- Phone
- Experience
- Languages
- Rate
- Availability
- Status

Assignment:

- Lead guide
 - Assistant guide
-

41. CORPORATE TOUR MANAGEMENT

Corporate request:

- Organization
- Contact person
- Email
- Phone
- Participants
- Destination
- Date
- Duration
- Budget
- Requirements

Workflow:

INQUIRY

↓

PROPOSAL

↓

NEGOTIATION

↓

CONFIRMED

↓

IN_PROGRESS

↓

COMPLETED

42. CUSTOM TOUR MANAGEMENT

Custom request:

- Destination
- Date
- Travelers
- Duration
- Budget
- Transport
- Hotel
- Food
- Activities
- Requirements

Convert:

Request
→ Quotation
→ Booking

43. QUOTATION ENGINE

Quotation:

- Customer
- Organization
- Tour
- Travelers
- Transport
- Hotel
- Food
- Activities
- Extra costs
- Discount
- Tax
- Grand total
- Validity
- Terms

Statuses:

- Draft
 - Sent
 - Viewed
 - Accepted
 - Rejected
 - Expired
-

44. PAYMENT MANAGEMENT

Payment:

- ID
- Booking
- Amount
- Method
- Transaction ID
- Reference

- Status
- Date
- Notes

Methods configurable.

45. PARTIAL PAYMENT

Example:

Total: 20,000

Paid: 8,000

Due: 12,000

Multiple payments allowed.

Historical payment records cannot be edited destructively.

Use reversal/correction transactions.

46. PAYMENT PROVIDER ABSTRACTION

Interface:

PaymentProvider

initiatePayment()

verifyPayment()

handleWebhook()

refundPayment()

Providers:

- Manual
- SSLCommerz
- bKash
- Nagad
- Stripe

Credentials only through environment/secure settings.

47. WEBHOOK PROCESSING

Handle:

- Success
- Failure
- Cancel
- Pending
- Refund

Webhook must be:

- Authenticated
- Idempotent
- Logged
- Server verified

Duplicate webhook must not duplicate payment.

48. INVOICE

Invoice:

- Invoice number
- Booking
- Customer
- Items
- Add-ons
- Discount
- Tax/fee
- Total
- Paid
- Due
- Status

Support PDF generation.

49. RECEIPT

Payment receipt:

- Receipt number
 - Booking
 - Customer
 - Amount
 - Method
 - Transaction ID
 - Received by
 - Date
-

50. REFUND ENGINE

Refund:

- Booking
- Payment
- Amount
- Reason
- Method
- Status
- Approved by
- Processed by

Workflow:

REQUESTED
→ APPROVED
→ PROCESSING
→ COMPLETED

51. CANCELLATION POLICY ENGINE

Policies can define:

- Cancellation window
- Refund percentage
- Fixed fee
- No-refund period

Policy is stored with booking snapshot.

Historical bookings must remain consistent even if policy changes later.

52. COUPON ENGINE

Coupon:

- Code
- Discount type
- Value
- Minimum order
- Maximum discount
- Usage limit
- Customer limit
- Start date
- End date
- Applicable tours
- Applicable categories
- Status

Track coupon usage.

53. EXPENSE MANAGEMENT

Expense:

- ID
 - Category
 - Amount
 - Date
 - Tour
 - Schedule
 - Vendor
 - Payment account
 - Reference
 - Receipt
 - Description
 - Created by
 - Approved by
-

54. EXPENSE APPROVAL

Optional approval workflow:

DRAFT

→ SUBMITTED

→ APPROVED

→ PAID

High-value expense threshold configurable.

55. VENDOR MANAGEMENT

Vendor:

- Name
- Type
- Contact
- Email
- Address
- Bank/payment information
- Status
- Notes

Types:

- Transport
 - Hotel
 - Food
 - Boat
 - Guide
 - Marketing
 - Other
-

56. ACCOUNTS PAYABLE

Track:

Vendor Bill

→ Total

→ Paid

→ Due

Support payment against vendor bill.

57. ACCOUNTS RECEIVABLE

Track:

Booking

→ Total

→ Paid

→ Outstanding

Customer receivable aging:

- Current
 - 1–7 days
 - 8–30 days
 - 31–60 days
 - 60+ days
-

58. CHART OF ACCOUNTS

Implement basic accounting structure.

Account types:

ASSET

LIABILITY

EQUITY

REVENUE

EXPENSE

Examples:

Cash

Bank

bKash

Nagad

Accounts Receivable

Accounts Payable

Tour Revenue

Add-on Revenue

Transport Expense

Hotel Expense

Food Expense

Marketing Expense
Salary Expense

Tenant-specific chart of accounts can be supported.

59. DOUBLE-ENTRY LEDGER

Financial transactions should support:

Debit
Credit

Example booking payment:

Debit: Cash
Credit: Tour Revenue / Receivable

Expense:

Debit: Transport Expense
Credit: Cash

Refund:

Debit: Refund / Revenue Adjustment
Credit: Cash

The ledger becomes the source of truth for financial reporting.

60. CASH MANAGEMENT

Track:

- Opening balance
- Cash income
- Cash expense
- Transfers
- Closing balance

Multiple cash accounts supported.

61. BANK ACCOUNT MANAGEMENT

Tenant can add:

- Bank name
- Account name
- Masked account number
- Currency
- Opening balance
- Status

Track deposits, withdrawals, transfers.

Never expose full sensitive banking information unnecessarily.

62. MOBILE WALLET MANAGEMENT

Support configurable accounts:

- bKash
- Nagad
- Rocket
- Other

Track:

- Incoming
 - Outgoing
 - Balance
 - Transaction reference
-

63. INTERNAL TRANSFERS

Support:

Cash → Bank

Bank → bKash

bKash → Bank

Bank A → Bank B

Transfer must create balanced accounting entries.

64. PROFIT & LOSS

Formula:

Gross Revenue

- Discounts

- Refunds

= Net Revenue

- Operating Expenses

= Net Profit

Reports by:

- Tour
- Schedule
- Destination
- Month
- Year
- Custom date range

65. TOUR PROFITABILITY

For every schedule:

Revenue

Expenses

Profit

Profit Margin

Occupancy

Average Booking Value

66. COMMISSION MANAGEMENT

Support:

- Staff commission
- Agent commission
- Referral commission

- Sales commission

Rules:

- Percentage
- Fixed
- Revenue-based
- Profit-based

Track:

Calculated

Approved

Payable

Paid

67. PAYROLL — OPTIONAL MODULE

Staff:

- Salary
- Bonus
- Commission
- Advance
- Deduction

Payroll:

Basic

+ Bonus

+ Commission

- Deduction

- Advance

= Net Salary

68. LEAD MANAGEMENT

Lead:

- Name
- Phone
- Email
- Source
- Interested tour

- Budget
- Assigned staff
- Status
- Notes

Statuses:

NEW
CONTACTED
INTERESTED
QUOTATION
FOLLOW_UP
CONVERTED
LOST

69. FOLLOW-UP MANAGEMENT

Create follow-up:

- Lead/customer
- Date
- Staff
- Note
- Reminder
- Status

Scheduled reminders through BullMQ.

70. SALES PIPELINE

Pipeline:

Lead
↓
Contacted
↓
Interested
↓
Quotation
↓
Negotiation
↓
Booking

↓
Paid

Track conversion rate.

71. AGENT / RESELLER MANAGEMENT

Future-ready module:

- Agent
 - Login
 - Commission
 - Booking
 - Balance
 - Credit limit
 - Payments
 - Ledger
-

72. REFERRAL SYSTEM

Referral code:

REF-SHANTO

Track:

- Referrer
 - Referred customer
 - Booking
 - Reward
 - Commission
-

73. CUSTOMER LOYALTY

Optional:

- Points
- Tiers

- Rewards
- Discounts

Tiers:

Standard

Silver

Gold

VIP

74. NOTIFICATION ENGINE

Channels:

- In-app
- Email
- SMS
- WhatsApp

Events:

- Booking created
 - Booking confirmed
 - Payment received
 - Payment due
 - Cancellation
 - Refund
 - Tour reminder
 - Low seats
 - Quotation
 - Follow-up
-

75. NOTIFICATION TEMPLATE ENGINE

Variables:

{{customerName}}
{{bookingId}}
{{tourName}}
{{tourDate}}
{{amount}}
{{dueAmount}}
{{pickupPoint}}

Templates tenant-configurable.

76. JOB / QUEUE SYSTEM

Use:

Redis
+
BullMQ

Queues:

notifications
emails
sms
whatsapp
payments
reports
pdf
seat-expiry
reminders

Workers must be retryable and idempotent.

77. SCHEDULED JOBS

Examples:

- Expire seat holds
 - Close expired booking schedules
 - Expire coupons
 - Expire quotations
 - Payment reminders
 - Tour reminders
 - Subscription renewal
 - Failed payment retry
 - Cleanup temporary files
-

78. WEBSITE CMS

Manage:

- Hero
- About
- Services
- Featured tours
- Destinations
- Gallery
- FAQ
- Testimonials
- Footer
- Contact
- Social links
- Announcements

All content tenant-specific.

79. PUBLIC BOOKING ENGINE API

Public endpoints:

GET /api/v1/public/tours
GET /api/v1/public/tours/:slug
GET /api/v1/public/schedules/:id
POST /api/v1/public/bookings
POST /api/v1/public/coupons/validate
GET /api/v1/public/booking/:reference

Only expose public-safe information.

80. CUSTOMER BOOKING LOOKUP

Customer can enter:

Booking Reference
+
Phone

Then retrieve:

- Booking status
 - Tour
 - Date
 - Travelers
 - Payment
 - Due
 - Pickup
 - Contact
-

81. PUBLIC AVAILABILITY

Expose:

- Schedule
- Capacity
- Availability

Optional:

32 / 50 booked
18 available

Tenant can disable public capacity visibility.

82. MEDIA MANAGEMENT

Support:

- Upload
- Delete
- Replace
- Search
- Tags
- Folder
- Tour association
- Gallery association

Storage provider abstraction.

83. REVIEW / TESTIMONIAL MANAGEMENT

Customer review:

- Customer
- Tour
- Rating
- Review
- Photos
- Approval
- Featured

Only approved reviews appear publicly.

84. CONTACT / INQUIRY MANAGEMENT

Contact submission:

- Name
 - Phone
 - Email
 - Subject
 - Message
 - Source
 - Status
 - Assigned staff
-

85. TOUR OPERATIONS

Operational checklist:

Vehicle Assigned
Driver Assigned
Guide Assigned
Hotel Confirmed
Food Confirmed
Traveler List Ready
Payments Checked
Pickup Confirmed

Departure Completed
Return Completed

86. TOUR OPERATIONS BOARD

Status:

UPCOMING
PREPARING
READY
RUNNING
COMPLETED
CANCELLED

87. INCIDENT MANAGEMENT

Incident:

- Tour
- Schedule
- Traveler
- Type
- Severity
- Location
- Description
- Action taken
- Staff
- Status

Types:

- Vehicle
 - Hotel
 - Weather
 - Traveler
 - Medical
 - Safety
 - Other
-

88. DOCUMENT MANAGEMENT

Documents:

- Invoice
- Receipt
- Quotation
- Agreement
- Hotel confirmation
- Transport document
- Internal document

Permission-controlled access.

89. REPORTING ENGINE

Reports:

Booking

- Total bookings
- Confirmed
- Cancelled
- Completed
- Source

Sales

- Revenue
- Conversion
- Average booking value

Customer

- New
- Returning
- VIP
- Lifetime value

Tour

- Occupancy
- Revenue
- Profit
- Cancellation

Finance

- Revenue
 - Expense
 - Profit
 - Receivable
 - Payable
 - Cash
 - Bank
-

90. ANALYTICS API

Dashboard endpoints:

GET /dashboard/overview

GET /dashboard/revenue

GET /dashboard/bookings

GET /dashboard/customers

GET /dashboard/tour-performance

GET /dashboard/financial-summary

GET /dashboard/upcoming-tours

Filters:

- Today
 - 7 days
 - 30 days
 - 3 months
 - Year
 - Custom range
-

91. SEARCH ENGINE

Global search:

Bookings

Customers

Travelers

Tours

Payments

Invoices

Expenses

Leads

Search by:

- Name
 - Phone
 - Email
 - Booking ID
 - Invoice
 - Transaction ID
-

92. AUDIT LOGGING

Track:

- User
- Organization
- Action
- Module
- Entity
- Entity ID
- Old data
- New data
- Timestamp
- IP
- User agent

Actions:

CREATE
UPDATE
DELETE
LOGIN
LOGOUT
CONFIRM
CANCEL
PAY
REFUND
APPROVE

93. SOFT DELETE

Do not permanently delete critical business data.

Use:

deletedAt
deletedBy

For:

- Customers
- Tours
- Bookings
- Expenses
- Vendors
- Users

Financial records should generally be immutable rather than deleted.

94. IDEMPOTENCY

Critical APIs must support idempotency.

Especially:

- Payment
- Webhook
- Booking creation
- Refund
- Invoice generation
- Subscription billing

Example:

Idempotency-Key

Duplicate requests must not create duplicate records.

95. API VERSIONING

Start:

/api/v1

All modules follow consistent versioning.

96. API RESPONSE STANDARD

Use consistent response structure.

Success:

```
{
  "success": true,
  "data": {},
  "message": "Success"
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "BOOKING_SEAT_UNAVAILABLE",
    "message": "Not enough seats available"
  }
}
```

97. PAGINATION

All large collections must support:

- Page
- Limit
- Sort
- Search
- Filter

Prefer cursor pagination for very large datasets where appropriate.

98. FILTERING

Support filters by:

- Date
- Status
- Tour
- Schedule

- Customer
 - Staff
 - Payment
 - Source
 - Destination
-

99. FILE EXPORT

Support:

- CSV
- Excel
- PDF

For:

- Bookings
- Travelers
- Payments
- Expenses
- Reports
- Manifest

Large exports should be generated asynchronously.

100. BACKGROUND PDF GENERATION

Use worker queue for:

- Invoice
- Receipt
- Quotation
- Manifest
- Reports

Store generated file securely.

101. API DOCUMENTATION

Use Swagger/OpenAPI.

Every endpoint must document:

- Authentication
 - Request body
 - Parameters
 - Response
 - Errors
 - Permissions
-

102. TESTING STRATEGY

Backend must include:

Unit Tests

- Pricing
- Coupon
- Seat calculation
- Refund
- Profit calculation
- Ledger
- Permission

Integration Tests

- Booking
- Payment
- Refund
- Expense
- Accounting

E2E Tests

Customer

→ Booking

→ Payment

→ Confirmation

→ Manifest

→ Tour Completion

→ Profit

103. CRITICAL BUSINESS TESTS

Must test:

Double booking

Two users attempt final seat simultaneously.

Expected:

Only one succeeds.

Duplicate payment webhook

Expected:

Only one payment.

Duplicate booking request

Expected:

Only one booking.

Cancellation

Expected:

Seats released.

Refund

Expected:

Financial records updated.

Expense

Expected:

Tour profit changes.

104. DATABASE ARCHITECTURE

Core entities:

organizations
organization_settings

users
roles
permissions
role_permissions
user_roles
sessions

subscriptions
subscription_plans
subscription_invoices
feature_entitlements

destinations
tour_categories
tour_packages
tour_schedules
tour_itineraries
tour_addons
tour_inclusions
tour_exclusions

customers
travelers

bookings
booking_travelers
booking_addons
booking_status_history
seat_holds
seat_allocations

pickup_points

vehicles
drivers
vehicle_assignments

hotels
hotel_rooms
room_allocations

guides
guide_assignments

corporate_requests

custom_tour_requests
quotations

payments
payment_transactions
payment_webhooks
invoices
invoice_items
receipts
refunds

coupons
coupon_usage
promotions

vendors
vendor_bills
vendor_payments

expense_categories
expenses
expense_approvals

accounts
account_transactions
journal_entries
journal_entry_lines
cash_accounts
bank_accounts
wallet_accounts
fund_transfers

commissions
commission_rules
commission_payments

leads
lead_activities
follow_ups

notifications
notification_templates

media
documents

reviews
contact_inquiries

tour_operations
tour_checklists
tour_incidents

website_pages
website_sections
faqs
testimonials

audit_logs

api_keys
webhook_endpoints

usage_records

105. IMPORTANT FINANCIAL DATA RULE

Financial history must be immutable.

Do NOT simply edit:

payment.amount

after accounting has posted it.

Instead use:

reversal
adjustment
refund
correction

This preserves financial integrity.

106. BOOKING PRICE SNAPSHOT

At booking time store:

- Base price

- Adult price
- Child price
- Add-on price
- Discount
- Coupon
- Tax/fee
- Total

If tour price changes tomorrow, old bookings remain unchanged.

107. POLICY SNAPSHOT

At booking time store relevant:

- Cancellation policy
- Refund policy
- Booking terms

Policy changes later must not change historical bookings.

108. TENANT DATA ISOLATION

Every query must be scoped:

WHERE organizationId = currentOrganization.id

Never allow:

GET /bookings/:id

to return another organization's booking.

Always validate ownership.

109. SAAS USAGE LIMITS

Track:

- Number of users

- Number of tours
- Number of bookings
- Storage
- Notifications
- API calls
- Branches

Usage can be tied to subscription plans.

110. API KEY MANAGEMENT

Enterprise tenants may generate API keys.

Support:

- Create
- Revoke
- Rotate
- Scope
- Last used
- Expiry

Permissions:

read:tours
read:bookings
write:bookings
read:customers

111. WEBHOOK MANAGEMENT

Tenant can configure webhook endpoints.

Events:

booking.created
booking.confirmed
booking.cancelled

payment.success
payment.failed

tour.created

tour.updated
tour.completed

Webhook delivery:

- Signature
 - Retry
 - Attempt count
 - Response status
 - Delivery log
-

112. BRANCH MANAGEMENT

Enterprise feature.

One organization:

Head Office

- └─ Barishal Branch
- └─ Dhaka Branch
- └─ Chittagong Branch

Users can be assigned to branches.

Bookings/expenses/revenue can be branch-specific.

113. MULTI-CURRENCY

Bangladesh-first:

BDT

Future:

USD
EUR
GBP
SAR
AED

Financial records should store currency and exchange rate where necessary.

114. TIMEZONE

Default:

Asia/Dhaka

Store timestamps in UTC where appropriate.

Display according to tenant timezone.

115. CONFIGURATION SYSTEM

Avoid hardcoded business rules.

Configurable:

- Currency
 - Tax
 - Fees
 - Cancellation
 - Booking deadline
 - Seat visibility
 - Coupon rules
 - Commission
 - Invoice numbering
 - Booking numbering
 - Payment methods
 - Notification providers
-

116. DEFAULT BANGLADESH SETTINGS

Initial tenant setup may default to:

Country: Bangladesh

Currency: BDT

Timezone: Asia/Dhaka

Language: Bengali

But everything must remain configurable.

117. BANGLADESH PAYMENT READINESS

Architecture should be prepared for:

- bKash
- Nagad
- SSLCommerz
- Bank transfer
- Cash

Do not fake gateway transactions.

Credentials are tenant/provider configuration.

118. COMMUNICATION READINESS

Architecture should support:

- Email
- SMS
- WhatsApp

Provider abstraction:

EmailProvider
SmsProvider
WhatsAppProvider

119. PLATFORM SUPPORT SYSTEM

Future SaaS feature:

- Support tickets
- Tenant issue
- Priority
- Assigned support agent
- Status
- Internal notes
- Attachments

120. PLATFORM ANALYTICS

SaaS owner can see:

- Total organizations
 - Active organizations
 - Trial organizations
 - MRR
 - ARR
 - Churn
 - New subscriptions
 - Bookings across platform
 - Platform revenue
 - Storage usage
 - Notification usage
-

121. SAAS BILLING METRICS

Track:

- MRR
 - ARR
 - ARPU
 - Churn
 - Trial conversion
 - Customer acquisition
 - Subscription upgrades
 - Downgrades
 - Failed payments
-

122. SYSTEM HEALTH

Platform admin:

- API health
- Database health
- Redis health
- Queue health

- Worker status
 - Error rate
 - Response time
-

123. OBSERVABILITY

Use structured logging.

Track:

- Request ID
- User ID
- Organization ID
- Endpoint
- Response time
- Status
- Error

Recommended:

- Sentry
 - OpenTelemetry
 - Prometheus/Grafana later
-

124. RATE LIMITING

Different limits:

Public:

- Booking attempts
- OTP
- Contact forms

Authenticated:

- API requests

Payment:

- Strict rate limit

Admin:

- Higher limits
-

125. ANTI-SPAM

Public endpoints:

- CAPTCHA/Turnstile if required
 - Rate limiting
 - IP throttling
 - Phone/email verification
 - Duplicate submission detection
-

126. BACKUP & RECOVERY

Database:

- Automated backups
- Point-in-time recovery where available

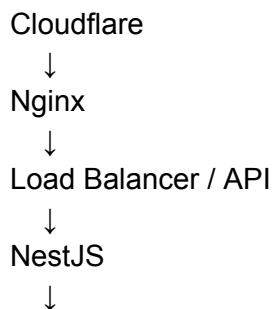
Storage:

- Backup
- Versioning where supported

Critical financial records must never depend on a single backup.

127. DEPLOYMENT ARCHITECTURE

Production:



PostgreSQL



Redis



BullMQ Workers

Storage:

Supabase Storage / S3

128. ENVIRONMENT MANAGEMENT

Environments:

development

staging

production

Separate:

- Database
 - Redis
 - Storage
 - Payment credentials
 - Email credentials
 - API keys
-

129. CI/CD

Pipeline:

Git Push



Lint



Type Check



Unit Test



Integration Test



Build



Migration Check

↓

Deploy

↓

Health Check

130. DATABASE MIGRATION POLICY

Use Prisma migrations.

Never manually modify production schema without migration tracking.

Every migration must be reviewed.

Production:

```
prisma migrate deploy
```

131. SEED DATA

Development/staging may contain:

- Demo organization
- Demo admin
- Demo tours
- Demo schedules
- Demo customers
- Demo bookings
- Demo expenses

Production must NOT contain fake business transactions.

132. INITIAL TOUR GROUP BARISHAL DATA

For the initial demo tenant, only use confirmed information.

Known tours:

Sundarbans

Price: ₳999

Date: 3 September 2026

Contact: 01739433221

Tanguar Haor

Price: ₳1599

Date: 20 August 2026

Departure: Natullabad

Departure timing: Thursday evening

Contact:

01748572189

01400441060

Do not invent additional business details.

133. BACKEND MODULE STRUCTURE

Recommended NestJS architecture:

```
src/
├── app.module.ts
├── common/
│   ├── guards/
│   ├── decorators/
│   ├── interceptors/
│   ├── filters/
│   ├── pipes/
│   ├── middleware/
│   ├── utils/
│   └── constants/
├── config/
└── database/
```


- auth/
- organizations/
- users/
- roles/
- permissions/
- subscriptions/
- destinations/
- tours/
- schedules/
- itineraries/
- pricing/
- addons/
- customers/
- travelers/
- bookings/
- seats/
- transport/
- hotels/
- guides/
- pickups/
- operations/
- corporate/
- custom-tours/
- quotations/
- leads/
- followups/
- payments/
- invoices/
- receipts/
- refunds/
- expenses/
- vendors/
- accounting/
- commissions/
- notifications/
- media/
- cms/
- reviews/
- reports/
- analytics/

- audit/
- webhooks/
- integrations/
- health/

134. SERVICE LAYER

Do not put all logic inside controllers.

Architecture:

Controller

↓

Service

↓

Domain / Business Logic

↓

Repository / Prisma

↓

PostgreSQL

Critical business logic should be isolated and testable.

135. TRANSACTION BOUNDARIES

Database transaction required for:

Booking

Create booking

+

Reserve seats

+

Create travelers

+

Create price snapshot

Payment

Verify payment

+

Create payment

+
Update booking balance
+
Create ledger transaction

Refund

Create refund
+
Update payment
+
Update booking
+
Ledger adjustment
+
Release seats if applicable

Expense

Create expense
+
Create payable
+
Ledger entry

136. API MODULE PRIORITY

Tier 1 — Absolutely Required

- Auth
- Organization
- Users
- Roles
- Tours
- Schedules
- Customers
- Travelers
- Bookings
- Seats
- Payments
- Invoices
- Expenses
- Accounting
- Reports

Tier 2 — Core Operations

- Transport
- Hotels
- Guides
- Pickup
- Operations
- Refund
- Coupons
- Quotations

Tier 3 — Business Growth

- CRM
- Leads
- Follow-up
- Corporate
- Custom tours
- Agents
- Commission
- Notifications

Tier 4 — SaaS / Enterprise

- Subscription
 - Billing
 - Usage
 - API keys
 - Webhooks
 - Branches
 - White-label
 - Custom domain
 - Platform analytics
-

137. DEVELOPMENT PHASES

PHASE 01 — Foundation

Build:

- NestJS
- TypeScript

- Prisma
 - PostgreSQL
 - Redis
 - Configuration
 - Logging
 - Error handling
 - Swagger
 - Docker
 - Environment management
-

PHASE 02 — SaaS Core

Build:

- Organization
 - Tenant isolation
 - User
 - Role
 - Permission
 - Subscription
 - Feature entitlement
-

PHASE 03 — Tour Engine

Build:

- Destination
 - Category
 - Tour package
 - Schedule
 - Pricing
 - Itinerary
 - Add-ons
 - Capacity
-

PHASE 04 — Booking Engine

Build:

- Customer
- Traveler

- Booking
 - Seat hold
 - Seat allocation
 - Booking status
 - Booking history
 - Pickup
-

PHASE 05 — Financial Core

Build:

- Payments
 - Payment providers
 - Invoice
 - Receipt
 - Refund
 - Expense
 - Vendor
 - Accounts
 - Ledger
 - Receivable
 - Payable
 - Cash
 - Bank
 - Wallet
 - Transfer
-

PHASE 06 — Operations

Build:

- Transport
 - Driver
 - Hotel
 - Room
 - Guide
 - Manifest
 - Tour operation
 - Incident
-

PHASE 07 — CRM & Sales

Build:

- Lead
 - Follow-up
 - Corporate
 - Custom tour
 - Quotation
 - Agent
 - Referral
 - Commission
-

PHASE 08 — Communication

Build:

- Notification
 - Email
 - SMS
 - WhatsApp
 - Templates
 - Queue
 - Scheduled jobs
-

PHASE 09 — CMS

Build:

- Website content
 - Gallery
 - Reviews
 - FAQ
 - Promotions
 - SEO
 - Public APIs
-

PHASE 10 — Analytics & Reports

Build:

- Dashboard
- Revenue

- Expense
 - Profit
 - Booking analytics
 - Customer analytics
 - Tour performance
 - Financial reports
-

PHASE 11 — Enterprise

Build:

- Branches
 - White-label
 - Custom domains
 - API keys
 - Webhooks
 - Usage limits
 - Advanced billing
-

138. MVP DEFINITION

The first sellable SaaS version should include:

Multi-Tenant
Authentication
Users
Roles

Tours
Schedules
Pricing
Capacity

Customers
Travelers
Bookings
Seats

Payments
Invoices
Refunds

Expenses
Basic Accounting
Profit/Loss

Transport
Hotels
Guides

CRM
Leads
Quotations

Notifications

Reports

Public Booking API

This is enough to onboard the first real travel businesses.

139. V2 FEATURES

After initial customers:

- Agent portal
 - Referral
 - Loyalty
 - Payroll
 - Branches
 - Advanced accounting
 - Payment gateway automation
 - WhatsApp automation
 - SMS automation
 - Custom domains
 - White-label
 - API marketplace
 - Advanced BI
-

140. FUTURE INTEGRATIONS

Architecture should be ready for:

Payment

- bKash
- Nagad
- SSLCommerz
- Stripe

Communication

- WhatsApp
- SMS
- Email

Accounting

- QuickBooks
- Xero
- Other accounting systems

Travel

- GDS
- Flight APIs
- Hotel APIs
- Maps
- Google Places

Do not build these integrations before validating the core product.

141. FINAL BACKEND SUCCESS CRITERIA

Backend is considered production-ready only when:

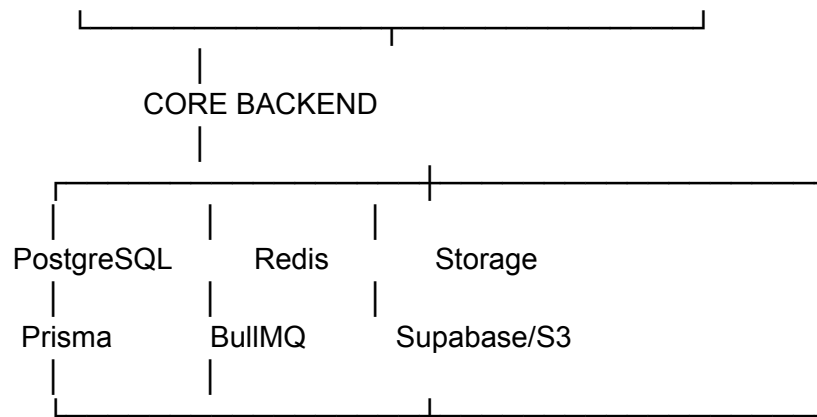
- Multi-tenancy works
- Tenant isolation is tested
- Authentication works
- RBAC works
- Tour management works
- Schedule management works
- Seat inventory is transaction-safe
- Booking works
- Price calculation is server-side
- Payment system works

- Webhooks are idempotent
- Invoice works
- Refund works
- Expense works
- Ledger works
- Profit/Loss works
- Operations work
- CRM works
- Notifications work
- Reports work
- Public booking API works
- Swagger documentation exists
- Tests exist
- Migrations work
- Backups are configured
- Logs exist
- Error monitoring exists
- No secrets are exposed
- No fake payment exists
- No fake financial data exists
- No cross-tenant data leakage exists

142. FINAL PRODUCT ARCHITECTURE

The completed product should ultimately look like:





143. PRODUCT POSITIONING

Do not position this simply as:

Tour Management Software

Position it as:

All-in-One Travel Business Operating System

Core promise:

Manage tours, bookings, customers, operations and finances from one platform.

Bangladesh-first positioning:

বাংলাদেশের **Tour Operator** ও **Travel Business**-এর জন্য সম্পূর্ণ **Business Management SaaS**।

144. MOST IMPORTANT ARCHITECTURAL PRINCIPLE

The backend must be designed so that:

One Company

today can become:

100 Companies

tomorrow.

And:

1,000 Companies

later.

Therefore:

- Multi-tenancy from day one
- Proper database relationships
- Transaction-safe booking
- Immutable financial records
- Role-based authorization
- Feature entitlements
- Queue-based background processing
- API versioning
- Audit logging
- Scalable PostgreSQL design

must be implemented from the beginning.

145. FINAL DEVELOPMENT ORDER

The actual implementation should proceed in this exact order:

01. Project Foundation



02. PostgreSQL + Prisma



03. Multi-Tenant Organization



04. Authentication



05. Users / Roles / Permissions



06. SaaS Plans / Entitlements



07. Destination / Category

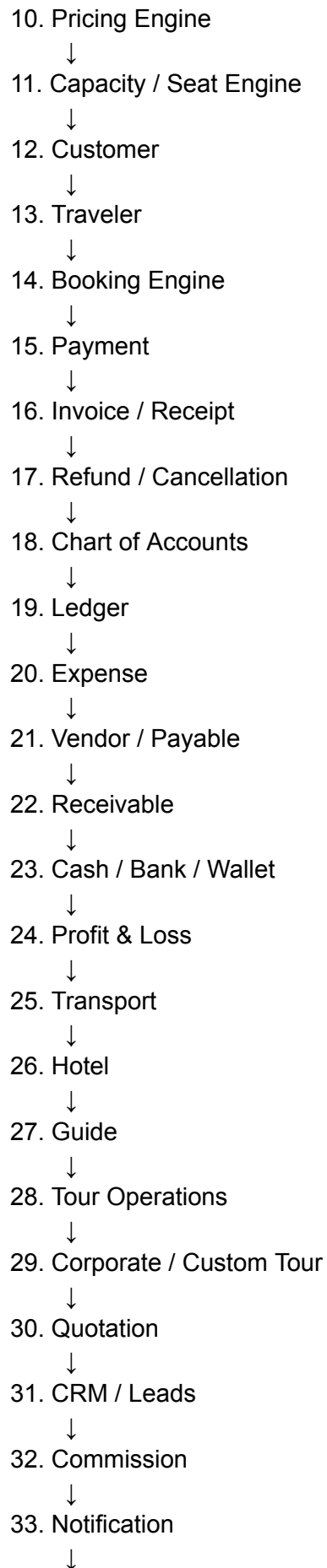


08. Tour Package



09. Tour Schedule





34. CMS / Public API
↓
35. Reports
↓
36. Analytics
↓
37. API Keys / Webhooks
↓
38. Branch / White Label
↓
39. Testing
↓
40. Security Audit
↓
41. Production Deployment

FINAL RULE

Do not start frontend/dashboard development until the core backend contracts are stable.

The backend should first expose clean APIs for:

AUTH
ORGANIZATION
USERS
ROLES
TOURS
SCHEDULES
PRICING
CAPACITY
CUSTOMERS
TRAVELERS
BOOKINGS
PAYMENTS
INVOICES
REFUNDS
EXPENSES
ACCOUNTING
OPERATIONS
CRM
QUOTATIONS
NOTIFICATIONS
REPORTS
CMS
SUBSCRIPTIONS

Then the web dashboard and public website become clients of this backend.

The final architecture must support:

One SaaS → Many Travel Companies → Separate Data → Separate Staff → Separate Tours → Separate Bookings → Separate Financials → One Scalable Platform.